Lecture 3 — MapReduce

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What is MapReduce?

MapReduce is a **programming model and execution framework** for processing massive datasets in parallel across clusters of machines. Introduced by Google (Dean & Ghemawat, OSDI 2004) and popularised by Apache Hadoop.

Core insight: Almost all big data tasks share a pattern — iterate over records → extract something → shuffle/sort → aggregate → output. MapReduce provides a clean abstraction for this via two operations: **Map** and **Reduce**. The key philosophy is *separating "what" from "how"*: you write the logic; the framework handles scheduling, parallelism, faults, and data movement.

2 The Three Phases

Map
Read each input record and emit intermediate (key, value) pairs

→

Shuffle & Sort
Group all values by key across all mappers; sort within each group

→

Reduce
For each key, aggregate its list of values and write final output

The Programmer's Interface

```
map    (k1, v1)       → List(k2, v2)
reduce (k2, List(v2)) → List(k3, v3)
```

You only write these two functions. The execution framework handles everything else: scheduling, sorting, data movement, and fault recovery.

3 Worked Example — Election Vote Count

Goal: count votes for Party A and Party B from multiple polling stations.

Step 1 · Map phase

Each mapper reads one vote and emits a pair: Party A → `(A, 1)`, Party B → `(B, 1)`. One map task calls the map function many times — once per record in its 128 MB split.

Step 2 · Shuffle & Sort

Framework groups all values by key across all mappers:
`A → [1,1,1,1,1,1]`
`B → [1,1,1]`
This happens distributed — data is NOT centralised.

Step 3 · Reduce phase

Each reducer receives one key and its list of values. Sums them: `A: 6`, `B: 3`. The reduce function is called once per distinct intermediate key.

4 MapReduce Implementation — Step by Step

1. **Submit:** User submits the program and configuration (e.g. number of workers) to the **Master node**. The Master coordinates only — it never processes data.
2. **Schedule:** Master assigns map and reduce tasks to available workers.
3. **Read & Map:** Input files are split into ~128 MB chunks. Each split = one map task. Workers apply the map function to each (key, value) pair in their split.
4. **Local Write (Shuffle starts):** Map workers write intermediate output to their own local disk, partitioned by key (one partition per reducer) and sorted by key within each partition.
5. **Remote Read (Shuffle continues):** Each reduce worker fetches its assigned partition from every map worker's local disk over the network.
6. **Reduce & Write:** Reducers process keys in sorted order, run the reduce function, and write output to HDFS (distributed file system).

Shuffle phase = steps 4 + 5. It spans both map and reduce workers. There is a **barrier** between map and reduce — all map tasks must finish before reduce tasks begin, though the shuffle can start copying data earlier.

5 Critical Terminology — Don't Confuse These!

Worker

A physical or virtual machine in the cluster. One worker handles multiple map tasks sequentially — once done with split 0, it may be reassigned to split 3.

Map Task

A unit of work: processing one 128 MB input split. It is *not* a machine. Number of map tasks = number of splits.

Map Function

A single call to your user-defined `map(k1, v1) → List(k2, v2)`. One map task calls this function once per (key, value) pair in the split — potentially thousands of times.

Reduce Task vs. Reduce Function

Reduce tasks: set by the user in configuration. Reduce function: called once per distinct intermediate key. A reduce task handles one or more keys, processed in sorted order.

Exam trap: "The map function is called once per input split" is **FALSE**. Map function calls = number of input (key, value) pairs. Map tasks = number of splits.

6 Split Size Trade-offs

Too large

Fewer splits → fewer map tasks → less parallelism. Cluster is underutilised.

Too small

Too many map tasks → Master overwhelmed by scheduling overhead. High coordination cost per task.

The standard Hadoop/HDFS split size is **128 MB** — a practical balance between parallelism and scheduling overhead.

7 Partition Step

The **Partitioner** determines which reduce worker handles each intermediate key. Default behaviour:

```
reducer_index = hash(key) mod num_reducers
```

Users can implement a **custom partitioner** to handle skew — when some keys have far more values than others, a custom partition distributes load more evenly. The partitioner runs during the local write phase (step 4).

> **Key fact:** The partitioner assigns keys to *reducers*, not mappers. "The partitioner determines which keys go to which mappers" is **FALSE**.

8 Combiner Step

Writing large volumes of intermediate data to disk and sending it over the network is expensive. A **Combiner** is a "mini-reducer" that pre-aggregates values locally on each map worker *before* the data is written to disk (during local write, step 4).

Without combiner

Mapper 1 emits: `A:1, A:1, A:1, B:1, B:1` → 5 records to disk and network.

With combiner

Mapper 1 emits: `A:3, B:2` → 2 records to disk and network. Reducers still get the correct result: `A: [3,1,2] → 6`.

> **Correctness rule:** The combiner must be safe to run 0, 1, or many times. It is only valid if the operation is both **associative** *and* **commutative**.
>
> ✓ Safe sum, max, min ✗ Unsafe mean (average), subtraction
>
> *Why mean fails:* `mean(mean(1,1), 2) = mean(1,2) = 1.5 ≠ mean(1,1,2) = 1.33`

9 Word Count — Three Versions

Version 0 · Naïve

For each word, emit `(word, 1)`. Huge intermediate output. High disk and network I/O. Combiners help significantly here by summing locally.

Version 1 · Per-line hash table

Maintain a hash table per line (per map function call). Emit aggregated counts per line. Fewer emits than V0. Combiners still provide additional benefit.

Version 2 · Per-split hash table

Hash table spans the entire split (via setup/cleanup hooks). Emit all counts once at the end. Minimum I/O. **Trade-off:** larger memory working set — risk of out-of-memory on diverse text.

10 Preserving State — Setup & Cleanup Hooks

The MapReduce API provides lifecycle hooks on each task object, allowing state to be maintained across many calls to the map/reduce function within a single task:

Hook	When called	Typical use
`setup()`	Once, before all map/reduce calls	Initialise hash tables, counters, external resources
`map() / reduce()`	Once per input pair / intermediate key	Core user logic
`cleanup()`	Once, after all map/reduce calls	Emit accumulated results, flush buffers, close resources

Word Count V2 uses `setup()` to initialise the hash table and `cleanup()` to emit all accumulated word counts at once per split.

11 Performance Guidelines

Linear scalability

Aim for algorithms where doubling data (or machines) proportionally scales cost. Avoid super-linear bottlenecks.

Minimise I/O

Prefer sequential over random disk access. Send data in bulk over the network, not in many small chunks.

Reduce working set

The "working set" is actively used memory per task. Large working sets risk out-of-memory errors at scale.

Use combiners wisely

Combiners cut disk and network I/O for free when the operation is associative and commutative.

12 Exam-Style Q&A

Q: How many times is the map function called?

> Once per input (key, value) pair — not once per split. Map tasks = number of splits. Map function calls = number of input pairs.

Q: How many times is the reduce function called?

> Once per distinct intermediate key. The number of reduce tasks is configured by the user — it is not the same as the number of intermediate keys.

Q: Does the partitioner determine which mappers process which keys?

> False. The partitioner determines which reducer handles each key. Mappers simply read their assigned input split.

Q: Is the shuffle phase run on a single node?

> False. The shuffle spans all map workers (local write) and reduce workers (remote read). It is inherently distributed.

Q: Can you always use the reducer as the combiner?

> Only if the operation is both associative and commutative (e.g. sum, max, min). Never for operations like mean or subtraction.

Q: What is the consequence of a split size that is too small?

> Too many map tasks → the Master is overwhelmed by scheduling overhead. Too large → limited parallelism and underutilised cluster.